

Implementando Uma Lista Sequencial Com Vetor de Capacidade Variável

(APP) Atividade Prática com Prazo de Entrega e Com Consulta – Peso 1

Objetivos Pedagógicos da Atividade

1. Compreender o modelo de lista sequencial com capacidade variável, diferenciando-o de uma lista sequencial de capacidade fixa e reconhecendo quando cada abordagem é mais adequada em termos de uso de memória e desempenho.
2. Relacionar a especificação do TAD de lista linear com uma implementação concreta baseada em vetor redimensionável, por meio da classe *ListaSequencialCapacidadeVariavel*, percebendo o papel do *array* interno e do controle de tamanho/capacidade;
3. Desenvolver a habilidade de projetar e implementar mecanismos de redimensionamento de vetores, planejando estratégias de expansão eficientes (por exemplo, crescimento proporcional) para evitar múltiplos realocamentos desnecessários durante inserções sucessivas;
4. Consolidar o entendimento de inserções em estruturas sequenciais, ao implementar o método *inserirEmLote(T[] novosElementos)* que: preserva a ordem de inserção dos elementos; adiciona todos os itens ao final da lista; e, ajusta automaticamente a capacidade quando necessário;
5. Exercitar boas práticas de programação defensiva, ao: tratar corretamente casos de entrada nula ou *array* vazio; garantir que o estado interno da lista permaneça consistente após a operação; evitar trabalho desnecessário quando não há elementos a inserir;
6. Estimular a capacidade de generalizar operações básicas para operações avançadas, entendendo a inserção em lote como uma composição/abstração de múltiplas inserções individuais, mas implementada de forma mais eficiente e coesa;
7. Fortalecer o domínio de recursos da linguagem Java na manipulação de *arrays*, incluindo manipulação de índices e controle explícito de tamanho versus capacidade.

Implementações

1. Implemente a classe concreta *ListaSequencialCapacidadeVariavel.java* (anexa a atividade) no *package estruturas.Lineares.estaticas.Lista* - valor (4,0) pontos.
2. Na classe *ListaSequencialCapacidadeVariavel.java*, complete o método com a seguinte assinatura:

public void inserirEmLote(T[] novosElementos)

Este método deve inserir múltiplos elementos de uma única vez no final da lista, garantido que a capacidade seja ajustada automaticamente caso necessário. Observações: Deve aceitar um *array* de elementos para inserção em lote (*batch*); Se o *array* for *null* ou vazio, o método deve retornar sem fazer alterações; Todos os elementos do *array* devem ser adicionados sequencialmente ao final da lista; A expansão deve ser eficiente, evitando múltiplos redimensionamentos desnecessários - valor (6,0) pontos.

Exemplo 01:

Vetor Inicial: [1, 2, 3] (tamanho = 3, capacidade = 8)

Novos Elementos: [4, 5, 6]

Vetor Final: [1, 2, 3, 4, 5, 6]

Obs.: No exemplo 01 não foi necessário o aumento de capacidade pois a capacidade inicial de 8 já atendia a entrada dos novos elementos e da capacidade do vetor final.

Exemplo 02:

Vetor Inicial: [1, 2, 3, 4] (tamanho = 4, capacidade = 6)

Novos Elementos: [5, 6, 7]

Vetor Final: [1, 2, 3, 4, 5, 6, 7]

Obs.: No exemplo 02 teve aumento da capacidade pois a capacidade inicial de 6 não atendia a entrada dos novos elementos e a capacidade do vetor final.

Resultado Esperado

Ao executar o **método main** da classe **AppListaSequencialCapacidadeVariavel** (anexa a atividade) o resultado deve ser parecido com a imagem abaixo:

```
1. Expansão automática de capacidade
  Capacidade inicial: 2
  Tamanho inicial: 0
  Após inserir lote de 5 elementos:
  Conteúdo: [10, 20, 30, 40, 50]
  Tamanho: 5
  Capacidade: 8

1.1 Verificação de elementos individuais
  Elemento na posição 0: 10
  Elemento na posição 2: 30
  Elemento na última posição: 50

2. Inserção em lista não vazia
  Estado inicial: [A, B]
  Após inserir lote de 4 elementos:
  Conteúdo: [A, B, C, D, E, F]
  Tamanho: 6

2.1 Array nulo (não deve alterar lista)
  Lista permanece inalterada: [A, B, C, D, E, F]

3. Array vazio (não deve alterar lista)
  Estado inicial: [1.5]
  Após inserir array vazio: [1.5]
```